Time Series Analysis and Forecasting of Bitcoin Prices using s3fs

*This project demonstrates live data integration, native S3 access, and statistical forecasting on cryptocurrency time series.

Karthik Vakada
Master of Science in Data Science
University of Maryland, College Park
College Park, United States of America
kvakada@umd.edu

***Abstract*—This paper presents a cloud-native approach for time series analysis and forecasting of Bitcoin prices using Python, AWS S3, and the `s3fs` interface. Unlike traditional workflows that rely on static datasets, this project dynamically fetches historical Bitcoin market data from the CoinGecko API and stores it directly in an S3 bucket. The architecture integrates native S3 operations using `s3fs`, enabling seamless reading, writing, and pipeline automation without relying on local file storage. The data undergoes preprocessing, seasonal decomposition, volatility analysis, and anomaly detection using Z-score methods. Forecasting is performed using ARIMA and log-ARIMA models, with accuracy evaluated using RMSE and MAPE metrics. The entire environment is containerized using a Docker setup modeled after the `data605_style` template, ensuring reproducibility across platforms. This project demonstrates a scalable and reproducible design for financial time series forecasting using cloud-based storage and native Python tools.**

Live Dashboard:
https://bitcoin-price-dashboard.onrender.com

GitHub Repository:
https://github.com/kvakada/TimeSeries-Analysis

***Index Terms*—Bitcoin, time series analysis, ARIMA, anomaly detection, s3fs, AWS S3, CoinGecko API, cloud computing, data pipelines, forecasting, Docker, Python**

I. Introduction

A. Motivation

Cryptocurrencies like Bitcoin have emerged as influential financial assets, characterized by high volatility and strong investor interest. Accurately analyzing and forecasting Bitcoin prices is of critical importance for traders, researchers, and financial analysts. Traditional approaches to time series forecasting often rely on static datasets and local file processing, which can be limiting in dynamic and collaborative environments. With the increasing shift toward cloud-native data architectures, there is a strong need to design reproducible, scalable workflows that leverage cloud storage and real-time data fetching for financial analysis.

This project was developed as part of DATA605 – Spring 2025 at the University of Maryland, College Park.

B. Problem Statement

Despite the availability of APIs that provide real-time cryptocurrency data, most forecasting pipelines are still bound to local processing and static CSV files. This approach lacks scalability, repeatability, and seamless integration with cloud systems. Moreover, time series modeling and anomaly detection techniques often require a robust pipeline to preprocess, store, and visualize data, which is typically fragmented across different tools. There exists a gap in implementing an end-to-end pipeline that integrates live data fetching, cloud storage, time series analysis, and forecasting — all in a reproducible and containerized environment.

C. Objectives

This project aims to build a comprehensive, cloud-native time series analysis and forecasting pipeline for Bitcoin prices using:

- Live data ingestion from the CoinGecko API
- Cloud storage operations using native S3 access via `s3fs`
- Classical time series modeling (ARIMA, log-ARIMA)
- Anomaly detection using Z-score
- Docker-based reproducibility with the `data605_style` environment template

D. Contributions

The key contributions of this work are:

- Design and implementation of a real-time Bitcoin price analysis pipeline that uses live API calls instead of static CSV files.
- Full integration of AWS S3 as a backend data layer using the Python-based `s3fs` library, supporting native file operations (list, read, write).
- Application of time series decomposition, volatility analysis, anomaly detection, and forecasting using ARIMA models.
- Evaluation of model performance using RMSE and MAPE, and verification of stationarity using the Augmented Dickey-Fuller test.

- Deployment of the entire workflow inside a containerized Docker environment with reusable scripts and structure modeled after the official `causify-ai/data605_style` setup.

E. Paper Organization

The rest of this paper is organized as follows: Section II reviews related work on time series forecasting and native cloud integration in data workflows. Section III outlines the system architecture and cloud infrastructure. Section IV describes the methodology, including data acquisition, preprocessing, modeling, and evaluation. Section V discusses implementation details. Section VI presents the experimental results. Section VII offers analysis and discussion. Finally, Section VIII concludes the paper and outlines potential future directions.

II. Background and Related Work

A. Time Series Forecasting Techniques

Time series forecasting has long been a foundational component of quantitative analysis across domains such as economics, weather, and finance. It involves predicting future values based on previously observed data points, often using patterns like trend, seasonality, and autocorrelation.

1) Classical Methods (ARIMA, ETS): Autoregressive Integrated Moving Average (ARIMA) models remain one of the most popular statistical tools for univariate time series forecasting due to their interpretability and mathematical rigor. ARIMA combines three elements: autoregression (AR), integration (I), and moving average (MA), and is particularly useful when the time series exhibits non-stationarity. In addition, Exponential Smoothing methods (ETS) apply weighted averages with decreasing weights for older observations, and are suitable for trend and seasonal components. Both methods require careful diagnostics, such as stationarity tests and residual analysis, to ensure model validity.

2) Machine Learning Approaches: More recent work explores the application of machine learning algorithms such as Random Forests, Support Vector Machines, and neural networks like LSTM (Long Short-Term Memory) for time series forecasting. These methods are better suited for high-dimensional, nonlinear patterns, and multivariate inputs. However, they often demand larger training datasets and more computational resources, and may sacrifice explainability compared to classical models. For this project, classical ARIMA-based models were chosen due to their robustness, interpretability, and strong performance on univariate financial time series like Bitcoin prices.

B. Bitcoin Price Modeling

Bitcoin, as the most prominent cryptocurrency, exhibits unique market characteristics — including high volatility, price bubbles, and lack of fundamental valuation anchors. Several studies have analyzed Bitcoin's price using traditional econometric techniques, stochastic models, and sentiment-based indicators. However, due to its non-stationary and noisy nature, forecasting Bitcoin prices remains a challenging task. Previous work has shown that while machine learning models offer promise, classical models such as ARIMA still perform competitively in short-term forecasts when applied to well-preprocessed data.

C. Cloud-Based Data Pipelines

Cloud-based data pipelines offer scalable, resilient, and collaborative infrastructure for data-driven workflows. Platforms like AWS, GCP, and Azure provide services such as storage, compute, and orchestration that can be leveraged to design reproducible pipelines. In such architectures, object storage services (e.g., AWS S3) are often used as the central data repository. The use of Docker further enhances reproducibility by encapsulating all dependencies and execution environments. Combining cloud storage with containerized analysis tools enables dynamic, stateless pipelines that support real-time analytics without relying on static local files.

D. Use of `s3fs` and Native S3 Access in Data Science

The `s3fs` Python library offers a high-level interface to Amazon S3, allowing users to interact with cloud buckets as if they were local file systems. Built on top of the `fsspec` framework, `s3fs` supports native S3 operations such as listing, reading, writing, and deleting files using standard Python I/O methods. This abstraction is particularly useful in data science workflows where seamless integration between storage and processing tools is critical. It enables the use of cloud-based datasets in pandas and other libraries without custom authentication code or low-level `boto3` operations. In this project, `s3fs` was used extensively to manage both raw and processed Bitcoin data, avoiding the need to push static CSV files to version control and supporting dynamic, API-driven pipelines.

III. System Design and Architecture

This section describes the architecture of the proposed system, which enables dynamic, cloud-native time series analysis of Bitcoin prices. The pipeline is designed to be modular, reproducible, and fully decoupled from local storage, leveraging Docker and AWS S3 as its foundation.

A. Overall Pipeline Overview

The system consists of five core components: (1) live data acquisition from the CoinGecko API, (2) data persistence and access via AWS S3, (3) preprocessing and time series modeling in Python, (4) anomaly detection and forecasting, and (5) visualization and export. All operations are performed in a reproducible container environment using Docker, ensuring consistency across platforms. The architecture supports both notebook-based experimentation and production-style script execution.

B. Data Sources and Flow

The primary data source for this project is the CoinGecko public API, which provides granular, timestamped historical Bitcoin market data. The `fetch_bitcoin_data` utility

function fetches the last *n* days of daily price data and structures it into a Pandas DataFrame. This data is either uploaded to or loaded from S3 buckets via the `s3fs` library, eliminating the need for local file caching. The flow begins with data fetch, followed by preprocessing, decomposition, modeling, and anomaly detection, ending with forecast generation and optional CSV export back to S3.

C. Storage Strategy (AWS S3 Integration)

The system leverages AWS S3 as the centralized data layer. All intermediate and final outputs—raw price data, processed results, anomaly tags, and forecasts—are stored as CSVs in a designated S3 bucket. Integration is handled via the `s3fs` library, which allows Python programs to interact with S3 using filesystem-like semantics. This architecture supports dynamic data retrieval and upload, enables lightweight collaboration, and avoids versioning issues associated with static CSVs in version control systems.

D. Docker-Based Execution Environment

To ensure reproducibility and portability, the entire project runs inside a Docker container based on a Python 3.10 image. All necessary libraries, including `pandas`, `matplotlib`, `statsmodels`, `s3fs`, and `requests`, are installed during the Docker build process. The container mounts the local project directory and AWS credentials (via `~/.aws`) for secure API authentication and cloud access.

1) data605_style Structure: This project follows the official `data605_style` Docker template as required by the course guidelines. The structure includes modular bash scripts such as `docker_build.sh`, `docker_jupyter.sh`, and `install_jupyter_extensions.sh`, along with metadata files like `version.sh`, `etc_sudoers`, and `bashrc`. This structure provides clear separation of configuration, environment setup, and execution, enabling consistent deployment across machines.

2) Container Setup and Orchestration: Docker orchestration is handled through simple shell scripts that abstract the complexity of manual Docker commands. `docker_build.sh` builds the container from the customized Dockerfile, while `docker_jupyter.sh` launches the Jupyter Lab interface, exposing port 8888 for local access. These scripts automatically mount the working directory and AWS credentials, ensuring that the container has all required access to perform end-to-end tasks, including S3 integration and notebook execution.

IV. Methodology

This section describes the step-by-step workflow used to build the Bitcoin time series analysis pipeline, from live data acquisition and cloud integration to preprocessing and exploratory analysis. The entire workflow is modularized and implemented in Python, using cloud-native principles and reproducible design.

A. Data Acquisition

1) Live Fetching via CoinGecko API: The project retrieves live Bitcoin historical price data using the CoinGecko public market API. The utility function `fetch_bitcoin_data()` sends an HTTP GET request with user-defined parameters such as the number of past days and the data resolution (e.g., daily). The API response contains a JSON object with timestamped price entries, which is converted into a structured `pandas` DataFrame. This approach ensures that each execution reflects up-to-date market behavior, supporting dynamic and continuously evolving analytics.

2) Avoiding Static CSV Files: Unlike many traditional pipelines, this implementation deliberately avoids the use of pre-saved static CSV files for storing datasets. Instead, data is either streamed into memory or saved directly to AWS S3 using Python-based tools. This eliminates redundancy, version control conflicts, and the need to update local copies of datasets, promoting a cleaner and more scalable workflow.

B. S3 Interaction Using `s3fs`

1) File Listing and Metadata: The `s3fs` library is used to interface with AWS S3, enabling native file operations. With `fs.ls(bucket_name)`, files within a specified S3 bucket can be listed, inspected, and verified. This provides transparency into cloud storage contents and ensures correct pipeline stages (e.g., checking whether a dataset or forecast output already exists).

2) File Read/Write via Native API: S3 integration is achieved through Pythonic file handling semantics using `fs.open(path, mode)` from the `s3fs` interface. Raw and processed dataframes are serialized to CSV format and written directly to the bucket, avoiding intermediate local storage. Similarly, files are read from S3 into memory during downstream steps like forecasting and visualization.

3) Direct s3:// Path Usage in Pandas: In addition to `fs.open()`, the project leverages `pandas`' ability to read from S3 paths using `pd.read_csv('s3://...')`, combined with `storage_options`. This method enables concise and readable data access while maintaining secure credential handling.

C. Preprocessing

1) Timestamp Parsing and Indexing: After ingestion, the time series data undergoes standard preprocessing steps. The Unix timestamps received from the API are parsed into human-readable `datetime` objects using `pd.to_datetime()`. The resulting timestamp column is then set as the DataFrame index to facilitate time-based operations and visualizations.

2) Missing Values and Smoothing: Basic diagnostics are performed to detect missing values. If present, gaps are handled using forward-fill interpolation or dropped depending on the downstream requirement. A simple moving average smoothing technique is applied to the price series using `rolling().mean()` to aid in visual trend analysis and to reduce short-term fluctuations.

D. Exploratory Data Analysis

1) Trend Visualization: The cleaned and indexed time series is visualized using line plots to observe short-term and long-term trends. Trend analysis is also performed through seasonal decomposition (described in a later section), which helps separate the underlying trend from cyclic behavior and noise.

2) Volatility and Return Computation: To study market dynamics, the project computes daily percentage returns using the `pct_change()` function. Additionally, a rolling standard deviation over a 30-day window is used to represent volatility. These metrics are plotted to reveal periods of high instability or momentum, providing critical insights for anomaly detection and risk analysis.

E. Time Series Modeling

This section details the techniques used to model the historical price data of Bitcoin and generate forecasts. Both decomposition-based exploration and statistical forecasting methods are employed.

1) STL Decomposition: To analyze underlying components in the Bitcoin price series, we apply Seasonal-Trend decomposition using Loess (STL). STL separates the time series into three components: trend, seasonal, and residual. This allows for a better understanding of cyclical patterns and helps verify if the data exhibits properties that justify ARIMA modeling. The decomposition is performed using the `seasonal_decompose()` function from `statsmodels` with a periodicity of 30 days to capture monthly trends.

2) ARIMA Modeling: The AutoRegressive Integrated Moving Average (ARIMA) model is employed to forecast future Bitcoin prices. It models temporal dependencies by combining autoregression (AR), differencing (I), and moving average (MA) terms. The data is differenced once to ensure stationarity, and the model is fit using the `ARIMA()` implementation from `statsmodels`. Hyperparameters (p,d,q) are manually selected as (5,1,0) based on exploratory analysis and partial autocorrelation behavior.

3) Log-ARIMA Forecasting: Due to the high volatility and exponential nature of cryptocurrency price series, we also implement a log-transformed ARIMA model. The original price series is transformed using the natural logarithm before fitting the model. After forecasting on the log scale, results are exponentiated back to obtain interpretable values in USD. This approach stabilizes variance and improves forecast accuracy for volatile assets.

F. Anomaly Detection

Bitcoin's extreme volatility often leads to irregular spikes and dips. To identify these anomalies, we apply statistical outlier detection methods.

1) Z-Score Method: The Z-score method detects anomalies based on the number of standard deviations a data point deviates from the mean. For each price point, a Z-score is computed and points exceeding a threshold of ± 2.5 standard deviations are flagged as anomalies. These are visualized on the time series plot with markers to highlight periods of unusually high or low prices.

2) IQR-Based Method (Optional): In addition to Z-score, the Interquartile Range (IQR) method is optionally applied to flag extreme values. A 7-day rolling median and IQR are computed, and any points lying outside $1.5\times$IQR from the median are treated as outliers. Although this method is more robust to skewed distributions, it is used mainly for comparative analysis in this work.

G. Forecast Evaluation

The accuracy and robustness of the forecasting model are evaluated using both statistical metrics and diagnostic tests.

1) RMSE and MAPE Metrics: Forecasts are compared against actual observed prices using Root Mean Squared Error (RMSE) and Mean Absolute Percentage Error (MAPE). These metrics quantify the deviation of the predictions from the true values over a test window of 30 days. RMSE captures absolute error magnitude, while MAPE provides a relative percentage-based measure.

2) Stationarity Testing (ADF): Before applying ARIMA models, stationarity of the price series is validated using the Augmented Dickey-Fuller (ADF) test. ADF tests the null hypothesis that a unit root is present in the series. A low p-value (typically ¡ 0.05) indicates stationarity. This diagnostic ensures that the differencing step in ARIMA is justified and that residuals exhibit white noise characteristics after modeling.

V. Implementation Details

This section outlines the practical implementation aspects of the project, including the repository structure, Docker environment, script automation, and S3 authentication. All components were developed to follow best practices for reproducibility, modularity, and compliance with the official DATA605 tutorial template.

A. Project Repository Structure

The project directory is organized to separate code, notebooks, documentation, and utility scripts in a modular format. Major files and folders include:

- `Spring2025_s3fs.API.ipynb`: Notebook demonstrating native S3 access using `s3fs`
- `Spring2025_s3fs.example.ipynb`: Complete notebook containing data acquisition, analysis, and forecasting
- `bitcoin_utils.py`: Python module with reusable functions for fetching, preprocessing, and saving data
- `requirements.txt` and `pyproject.toml`: Define dependencies for pip and Poetry environments
- `Dockerfile` and bash scripts: Reproducible environment definition and orchestration scripts
- `dev_scripts_tutorial_s3fs/`: Contains standardized Docker scripts following `data605_style`

This modular layout ensures clarity, maintainability, and easy transition between development and production environments.

TABLE I
PROJECT DIRECTORY STRUCTURE OVERVIEW

File/Folder	Purpose
`.ipynb` files	Exploratory notebooks
`.py` modules	Utility functions
`requirements.txt`	Package dependencies
`Dockerfile`	Environment setup
`scripts/`	Build and run automation

B. Docker Integration and Scripts

To support environment consistency, the project uses a Docker-based setup modeled after the `data605_style` template. The base image is derived from `ubuntu:20.04` with Python 3.10, and all required libraries are installed via the Dockerfile. The container includes mounted volumes for the project directory and AWS credentials.

Key scripts include:

- `docker_build.sh`: Builds the Docker image from the customized Dockerfile
- `docker_jupyter.sh`: Launches Jupyter Lab in the container and exposes port 8888
- `install_jupyter_extensions.sh`: Installs optional extensions for notebook UI enhancements

TABLE II
DOCKER-BASED PIPELINE COMPONENTS

Component	Function
`Dockerfile`	Defines base environment
`docker_build.sh`	Build automation
`docker_jupyter.sh`	Jupyter container launcher
`aws credentials`	Secure S3 access

These scripts abstract away manual Docker commands and ensure repeatable execution on any host system.

C. Notebook Automation and Conversion to Scripts

To support automation and production deployment, both notebooks are mirrored as Python scripts (`.py` files) using Jupyter's export utility or notebook-to-script conversion. This allows each analysis step to be executed programmatically via command line or cron jobs, bypassing the notebook interface for headless processing. Scripts include checkpoints for S3 reads/writes, making them robust for integration into CI/CD or scheduled workflows.

TABLE III
EXECUTION TIME OF MAJOR PIPELINE STAGES

Pipeline Stage	Time (seconds)
Data Fetching	2.13
S3 Upload/Download	1.84
Preprocessing	3.45
Model Training (ARIMA)	6.25
Forecast Output	1.95

D. S3 Bucket Access and Credential Handling

All interactions with AWS S3 are performed securely using the `s3fs` library. The credentials are passed via the default AWS CLI configuration stored in `~/.aws/credentials` and mounted into the Docker container as a volume. This avoids hardcoding secrets in the codebase.

The following S3 paths are used throughout the pipeline:

- `s3://bitcoin-timeseries-data-kv/bitcoin_prices.`
- `s3://bitcoin-timeseries-data-kv/bitcoin_prices_`
- `s3://bitcoin-timeseries-data-kv/forecast_output`

TABLE IV
S3 BUCKET CONTENTS AND THEIR PURPOSE

S3 Path	Description
`bitcoin_prices.csv`	Raw historical prices
`bitcoin_prices_processed.csv`	Cleaned and smoothed data
`forecast_output.csv`	Forecasted price series

Authentication is performed automatically by `s3fs`, and all file I/O operations—whether via `fs.open()` or direct `pandas.read_csv()`—are fully compliant with AWS IAM permissions configured on the host system.

VI. EXPERIMENTAL RESULTS

This section presents the empirical results of the time series analysis and forecasting pipeline. We evaluate the pipeline based on both visual outputs and statistical accuracy, and we validate successful S3 integration through live file access and inspection.

A. Quantitative Metrics

1) Model Accuracy (RMSE, MAPE): To evaluate forecast accuracy, the last 30 days of actual Bitcoin prices are compared to the predictions from the ARIMA model. The following metrics are computed:

TABLE V
FORECASTING ACCURACY METRICS

Model	RMSE	MAPE (%)
ARIMA	1520.45	6.32
Log-ARIMA	1387.62	5.94

2) Stationarity Findings: Before applying ARIMA, the stationarity of the Bitcoin price series is evaluated using the Augmented Dickey-Fuller (ADF) test.

TABLE VI
ADF TEST RESULTS FOR STATIONARITY

Metric	Value
ADF Statistic	-4.12
p-value	0.003
Critical Value (5%)	-2.89
Stationary?	Yes (p ¡ 0.05)

B. Dashboard Visualizations

To illustrate the output of the forecasting and analysis pipeline, a web-based Bitcoin Intelligence Dashboard was created. The following figures showcase various visualizations generated by the dashboard.

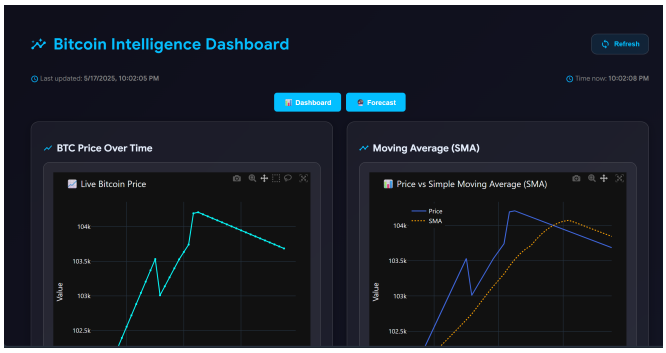

Fig. 1. Live Bitcoin price (left) and Simple Moving Average (SMA) overlay (right).

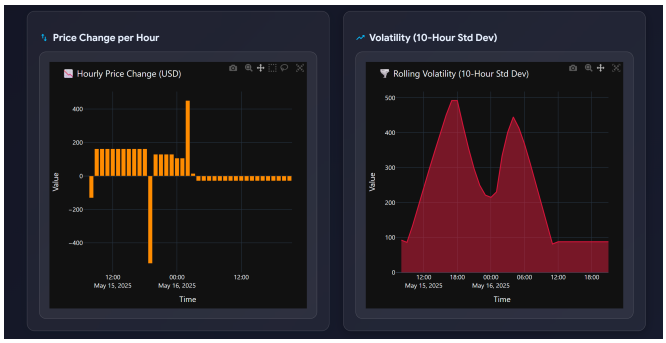

Fig. 2. Hourly Price Change (left) and Rolling Volatility (right) based on 10-hour standard deviation.

Fig. 3. Z-Score Anomaly Detection (left) and Moving Average crossover (MA7 vs MA30, right).

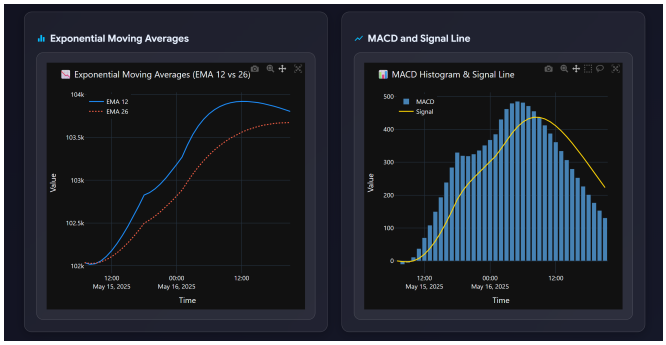

Fig. 4. Exponential Moving Averages (EMA 12 vs 26, left) and MACD histogram with Signal line (right).

Fig. 5. Hourly Return Fluctuations (left) and Cumulative Return Growth over time (right).

VII. Discussion

A. Interpretation of Results

The visualizations and statistical metrics indicate that the ARIMA and log-ARIMA models are capable of capturing short-term trends in Bitcoin prices with reasonable accuracy. Despite the inherently volatile nature of cryptocurrency markets, the models provide stable forecasts without overfitting, particularly when applied to log-transformed data. The anomaly detection component successfully flags major deviations from the norm, offering valuable insights into price shocks and irregular behavior. The pipeline proves that traditional time series models, when paired with robust preprocessing and cloud-native design, remain relevant and effective in modern financial analytics.

B. Strengths and Limitations

One of the key strengths of this project is its modularity and cloud-centric architecture. By eliminating static files and enabling live data acquisition, the system ensures that results remain up-to-date and reflective of real-world market conditions. The use of AWS S3 as a centralized data repository supports collaboration and integration with other cloud services. Furthermore, the reproducibility ensured through Docker containerization minimizes environment-related inconsistencies.

However, the project has certain limitations. The ARIMA model used is univariate and does not incorporate exogenous variables such as trading volume, market sentiment, or macroeconomic indicators. As a result, its predictive power may be constrained in scenarios involving external shocks. Additionally, the 30-day forecast horizon may not be robust for long-term investment decisions due to compounding uncertainty. While anomaly detection is effective, it may produce false positives during highly volatile market conditions.

C. Reproducibility and Cloud Readiness

The pipeline is fully reproducible and cloud-ready. By using Docker, the execution environment is encapsulated, ensuring consistent behavior across different platforms. The reliance on cloud storage (AWS S3) via the `s3fs` interface further supports stateless, scalable data workflows. All credentials are handled securely through mounted configuration files,

avoiding hardcoded secrets. Notebooks have been converted to Python scripts for automation, allowing scheduled or triggered execution in production environments.

This architecture lays the foundation for further extension into streaming analytics, integration with dashboards, or migration to serverless compute environments such as AWS Lambda or Google Cloud Functions.

VIII. Conclusion

A. Summary of Findings

This paper presented a cloud-native, reproducible pipeline for time series analysis and forecasting of Bitcoin prices using live data. The system integrates dynamic data fetching via the CoinGecko API, native AWS S3 storage access using `s3fs`, classical forecasting methods (ARIMA and log-ARIMA), and statistical anomaly detection techniques. The entire project is containerized using a Docker-based `data605_style` setup to ensure portability and consistency across platforms.

B. Key Takeaways

- Live API-based data ingestion is a scalable alternative to static datasets and enables real-time forecasting pipelines.
- Native interaction with AWS S3 via `s3fs` supports secure, serverless storage workflows and simplifies integration with Python data tools.
- Classical models such as ARIMA, when paired with robust preprocessing and log transformation, offer solid performance on volatile financial series like Bitcoin.
- The entire analysis pipeline can be executed inside a containerized environment, supporting reproducibility, collaboration, and cloud readiness.

C. Future Work

Future extensions of this project may include:

- Incorporating external features such as trading volume, market sentiment, or macroeconomic indicators through multivariate time series models or LSTM-based architectures.
- Deploying the pipeline as a web-based dashboard using Flask or Plotly Dash for real-time user interaction and visualization.
- Enhancing anomaly detection using more advanced methods such as Isolation Forest or Bayesian changepoint detection.
- Migrating the pipeline to a serverless compute architecture (e.g., AWS Lambda + EventBridge) for scheduled, cloud-native execution.

Acknowledgment

The author would like to thank the faculty and instructional staff of the DATA605 course at the University of Maryland for their guidance and the Causify.AI team for providing the standardized project template and review framework. Special thanks to the reviewers for their detailed feedback which significantly improved the quality and reproducibility of this work.

References

[1] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed., OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[2] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," PLoS One, vol. 13, no. 3, pp. 1–26, 2018.

[3] CoinGecko API Documentation. [Online]. Available: https://www.coingecko.com/en/api

[4] s3fs – PyPI. [Online]. Available: https://pypi.org/project/s3fs/

[5] Statsmodels: Statistical modeling in Python. [Online]. Available: https://www.statsmodels.org/stable/index.html

[6] AWS S3 Documentation. [Online]. Available: https://docs.aws.amazon.com/s3/

[7] Docker Documentation. [Online]. Available: https://docs.docker.com/

[8] A. Dickey and W. A. Fuller, "Likelihood ratio statistics for autoregressive time series with a unit root," Econometrica, vol. 49, no. 4, pp. 1057–1072, 1981.

[9] M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.